

Rhai

Evaluate expressions and rules stored as text



Install & load

```
INSTALL evalexpr_rhai FROM community;  
LOAD evalexpr_rhai;
```

01 At a glance

Run Rhai expressions inside a query, pass row values as JSON context, and read a typed success or error result. Useful for configurable business rules, small transformations, and scripts that need local variables or branching.

Script text



Row context



JSON result

02 SQL entry point + Rhai syntax

```
evalexpr_rhai(script)
```

Evaluate a script without row context.

```
evalexpr_rhai(script, context)
```

Expose a JSON object as context inside the script.

```
.ok
```

JSON result on success; NULL when the script fails.

```
.error
```

Error text on failure; NULL when the script succeeds.

```
context.field
```

Read a field supplied by the SQL query.

```
let / if / fn
```

Bind variables, branch, and define helpers inside scripts.

03 Keep in mind

Keep the boundary explicit

The result is UNION(ok JSON, error VARCHAR). Cast .ok to a SQL type before arithmetic or joins. Inspect .error to distinguish a failed script from a successful JSON null.

04 Evaluate and cast

```
SELECT evalexpr_rhai(  
  'fn sq(x) { x * x } sq(context.n)',  
  {'n': 7}  
) .ok::INTEGER AS square;  
-- 49
```

Define helpers in the script; return its final expression.

05 Apply one rule to each row

```
SELECT salary, evalexpr_rhai(  
  'context.salary ≥ 100000',  
  {'salary': salary}  
) .ok::BOOLEAN AS eligible  
FROM (VALUES (80000), (120000)) t(salary);  
-- false, true
```

Use a constant rule with a different JSON context for each row.

06 Read a failure without losing the query

```
SELECT evalexpr_rhai('1 / 0').error;  
-- Runtime error text
```

A failing Rhai expression returns the error arm.

07 In practice

Use it where rules change

Scripts run per row. Keep routine transformations in native SQL; use Rhai when stored rules or branching improve clarity. A constant script can reuse its parsed expression while context changes.

[Full docs & examples ↗](#) [Source ↗](#)